

107. Binary Tree Level Order Traversal II

PLATFORM

Platform: LeetCode

DIFFICULTY

MEDIUM

DATE

Solved: October 2, 2026

Problem Summary

Given the root of a binary tree, return the bottom-up level order traversal — from leaf level to root level, left to right within each level.

Approach

Standard BFS level order traversal (LC 102) with one extra step — reverse the result list at the end using `Collections.reverse()`.

Section

Time Complexity

$O(n)$ — every node visited once + $O(n)$ reverse = $O(n)$.

Space Complexity

$O(n)$ — queue holds at most one full level.

Approaches

- **Brute Force:** Normal level order → store all levels → reverse at end. $O(n)$ time $O(n)$ space.
- **Optimal:** Same as brute force here — reversing a list is $O(n)$ and unavoidable. No better approach exists.
- **Alternative:** Use `LinkedList` and `addFirst()` instead of reversing — inserts each level at the front directly.

Algo

1. If `root == null` return empty list.
2. Add root to queue.
3. While queue not empty:
 - Snapshot size = `q.size()`.
 - Create result list for current level.
 - Poll size nodes, add values to result, add non-null children to queue.
 - Add result to ans.
4. `Collections.reverse(ans)`.
5. Return ans.

Tree

107. Binary Tree Level Order Traversal II

Medium

Given the root of a binary tree, return the bottom-up level order traversal of its nodes' values. (i.e., from left to right, level by level from leaf to root).

Example 1:

```
graph TD
    3((3)) --> 9((9))
    3 --> 20((20))
    9 --> 15_9((15))
    20 --> 15_20((15))
    20 --> 7((7))
```

Input: root = [3,9,20,null,null,15,7]
Output: [[15,7],[9,20],[3]]

Example 2:

Input: root = [1]
Output: [[1]]

Example 3:

Input: root = []
Output: []

Accepted

34 / 34 testcases passed

Sahil Khan submitted at Oct 02, 2026 17:26

Unlock the Full LeetCode Experience

Runtime: 1 ms | Beats 98.15%

Memory: 44.56 MB | Beats 38.86%

Code | Java

```
1 /**
2  * Definition for a binary tree node.
3  * public class TreeNode {
4  *     int val;
5  *     TreeNode left;
6  *     TreeNode right;
7  *     TreeNode() {}
8  *     TreeNode(int val) { this.val = val; }
9  * }
```

Code

```
16 class Solution {
17     public List<List<Integer>> levelOrderBottom(TreeNode root) {
18         Queue<TreeNode> q = new LinkedList<>();
19         List<List<Integer>> ans = new ArrayList<>();
20
21         if(root==null) return ans;
22
23         q.offer(root);
24
25         while(!q.isEmpty()){
26             int size = q.size();
27             List<Integer> result = new ArrayList<>();
28
29             for(int i=0; i<size; i++){
30                 TreeNode curr = q.poll();
31
32                 if(curr.left!=null) q.offer(curr.left);
33                 if(curr.right!=null) q.offer(curr.right);
34
35                 result.add(curr.val);
36             }
37
38             ans.add(result);
39         }
40         Collections.reverse(ans);
41         return ans;
42     }
43 }
```

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1 Case 2 Case 3